RECOMMENDATION SYSTEMS

State-of-the-Art Deep Dive 2026

Collaborative Filtering Content-Based Two-Tower Embeddings LLM-Powered

5 Paradigms	**30+ Models**	**Production Patterns**	**Research Frontiers**

Prepared for Satish V. March 2026

Table of Contents

1. Introduction & Taxonomy 3

2. Collaborative Filtering 4

2.1 Memory-Based: User-User & Item-Item 4

2.2 Matrix Factorization (SVD, ALS, NMF) 4

2.3 Neural Collaborative Filtering (NCF) 5

2.4 Variational Autoencoders for CF 5

3. Content-Based Filtering 6

3.1 Classical TF-IDF & Feature Engineering 6

3.2 Deep Content Models & Foundation Encoders 6

4. Two-Tower Models 7

4.1 Architecture & Training Objectives 7

4.2 Negative Sampling Strategies 8

4.3 ANN Search & Production Deployment 8

4.4 SOTA Two-Tower Systems 9

5. Embedding-Based Recommendation 10

5.1 Item2Vec, Prod2Vec & Graph Embeddings 10

5.2 Session-Based (GRU4Rec, SASRec, BERT4Rec) 10

5.3 Graph Neural Networks for RecSys 11

6. LLM-Powered Recommendation 12

6.1 Prompting & In-Context Recommendation 12

6.2 Fine-Tuned LLM Recommenders 13

6.3 SOTA LLM RecSys Models 2025-2026 14

7. Hybrid & Multi-Stage Ranking 15

8. Multimodal Recommendation 17

9. Evaluation: Metrics, Benchmarks & Pitfalls 18

10. Production Architecture 20

11. Full SOTA Model Comparison 22

12. Research Frontiers 2026 24

1. Introduction & Taxonomy

Recommendation systems are among the highest-value ML applications deployed at scale, directly driving revenue at Netflix, YouTube, Amazon, Spotify, TikTok, and LinkedIn. By 2026 the field has undergone a fundamental transformation: architectures have evolved from matrix factorization to deep retrieval to LLM-native systems operating in real-time over billions of items.

This document covers every major paradigm with full depth: the mathematics, training objectives, known failure modes, and production deployment patterns. It is structured from classical foundations to cutting-edge SOTA, building intuition at each layer.

The Five Paradigms

Paradigm	Core Idea	Strength	Key Weakness
Collaborative Filtering	Users who agreed in the past will agree in the future	No item features needed; learns latent signals	Cold start, sparsity, popularity bias
Content-Based	Recommend items similar to what a user liked before	Works for new items; no user-user data needed	Feature burden; filter bubble risk
Two-Tower	Dual encoders for user and item trained on interactions	Scales to billions with ANN; highly flexible	Cross-feature interaction lost at inference
Embedding-Based	Learn dense reps from sequences or interaction graphs	Captures temporal patterns, social structure	Needs rich interaction history
LLM-Powered	Language model reasoning over items and user history	Generalises, explains, handles cold start natively	Latency, cost, hallucination risk

The Production Funnel

Every large-scale system uses a multi-stage funnel. Broad candidate retrieval narrows to a ranked shortlist, then re-ranked with a precision model before serving.

Stage	Scale	Typical Model	Optimisation Goal
Candidate Generation	10M to 1,000	Two-Tower / ANN / CF	Recall: do not miss relevant items
Coarse Ranking	1,000 to 200	Wide & Deep / DCN	Balance relevance and diversity
Fine Ranking	200 to 50	DLRM / Transformer	Maximise engagement or conversion
Re-Ranking	50 to 10-20	LLM / rule-based	Diversity, constraints, business rules
Serving	10-20 to user	Feature store and cache	Latency < 100ms, A/B assignment

2. Collaborative Filtering

Collaborative Filtering is the founding paradigm of recommendation. The core hypothesis: past behaviour reveals preferences, and users with similar behaviour will share future preferences. Every modern deep learning recommender is an extension of CF.

2.1 Memory-Based CF

User-User CF

Find users most similar to the target user, then aggregate their ratings on items the target has not yet seen.

```
pred(u,i) = mu_u + SUM_{v in N(u)} sim(u,v)*(r_vi - mu_v) / SUM sim(u,v)
```

* Similarity metrics: Pearson correlation handles rating-scale bias; cosine and Jaccard for implicit feedback
* Neighbourhood N(u): Top-k most similar users; k typically 20-100 and is a critical hyperparameter
* Weakness: O(|U|^2) similarity computation; does not scale beyond a few million users

Item-Item CF

Amazon's original algorithm (Linden et al. 2003). Precompute item-item similarities offline; at query time aggregate over items similar to what the user has consumed.

```
pred(u,i) = SUM_{j in N(i) intersect rated(u)} sim(i,j)*r_uj / SUM sim(i,j)
```

* Key advantage over user-user: Item space is more stable than user space; similarities can be precomputed
* Adjusted cosine: Subtract per-user mean to remove rating-scale bias; outperforms raw cosine
* SLIM (Sparse Linear Methods): Learn sparse item-item weight matrix W to maximise prediction accuracy

2.2 Matrix Factorization

The seminal insight from the Netflix Prize (2009): decompose the user-item rating matrix R into low-rank matrices U and V such that R approximates U * V^T. Each user and item gets a dense latent-factor vector.

```
min_{P,Q}  SUM_{(u,i) in obs} (r_ui - p_u * q_i^T)^2  +  lambda*(||p_u||^2 + ||q_i||^2)
```

SVD / FunkSVD

* Simon Funk's Netflix solution: SGD on observed entries only, not full matrix decomposition
* Bias extension: r_hat_ui = mu + b_u + b_i + p_u * q_i^T adds global mean, user bias, and item bias
* SVD++ (Koren 2008): Adds implicit item vectors for all items a user interacted with regardless of rating

ALS (Alternating Least Squares)

Fix Q, solve for P in closed form. Then fix P, solve for Q. Repeat. Preferred for implicit feedback and distributed training.

```
p_u = (Q^T C_u Q + lambda*I)^{-1} Q^T C_u r_u    where C_u = diag(1 + alpha * r_u)
```

* Implicit ALS (Hu et al. 2008): Confidence c_ui = 1 + alpha*r_ui where r_ui is count data such as plays or clicks
* Distributed: Spark MLlib ALS scales to billions of interactions; industry standard for large-scale CF
* Convergence: Each alternating step decreases loss; guaranteed convergence with no learning rate tuning

NMF (Non-Negative Matrix Factorization)

* Constraint U,V >= 0 forces additive parts-based decomposition; more interpretable latent factors
* Best for: Topic-like decompositions where factors have semantic meaning such as genre clusters

2.3 Neural Collaborative Filtering (NCF)

He et al. (2017) replace the dot product of MF with a multi-layer perceptron. Key insight: dot product assumes linear user-item interaction; MLP learns arbitrary interactions.

```
y_hat_ui = sigma( h^T * phi(MLP([e_u concat e_i])) )    concat = concatenation
```

* GMF branch: Element-wise product of user and item embeddings; generalised matrix factorization
* MLP branch: Concatenate embeddings and pass through FC layers; captures non-linear interactions
* NeuMF: Combine GMF and MLP outputs with a jointly trained final layer; best of both
* Training objective: Binary cross-entropy with negative sampling on unobserved interactions

KEY INSIGHT: NCF is primarily a ranking model, not a retrieval model. For billion-scale retrieval you need models that decompose into precomputable user and item vectors. That is the Two-Tower architecture.

2.4 Variational Autoencoders for CF

CDAE - Collaborative Denoising Autoencoders

* Input: Corrupted (randomly zeroed) user interaction vector
* Task: Reconstruct the full clean interaction vector; model learns to complete the preference profile
* User node: Each user gets a dedicated input node; user-specific bias baked into the encoder

Mult-VAE (Liang et al. 2018)

Apply VAE to CF with multinomial likelihood. One of the most principled probabilistic approaches to recommendation.

```
L = E_{q(z|x)}[log p(x|z)] - beta * KL(q(z|x) || p(z))
```

* Multinomial likelihood: More appropriate than Gaussian for implicit feedback such as click or no-click
* Beta annealing: Start beta=0 for pure reconstruction, anneal up to prevent posterior collapse
* RecVAE: Alternating encoder/decoder training with composite prior; current SOTA in autoencoder CF

3. Content-Based Filtering

Content-based methods recommend items similar to what a user has liked, based on item features rather than other users' behaviour. They are immune to the cold-start problem for new items and provide transparent explanations.

3.1 Classical TF-IDF & Feature Engineering

TF-IDF for Text Features

```
TF-IDF(t,d) = tf(t,d) * log(N/df(t))    User profile = SUM_i w_i * TF-IDF(i) over liked items
```

* Item representation: TF-IDF vectors over descriptions, tags, reviews, and metadata
* User profile: Weighted average of TF-IDF vectors of liked items; simple and interpretable
* Cosine similarity: Normalise vectors; dot product equals cosine similarity
* Limitation: No semantic understanding; car and automobile are treated as different features

Structured and Hybrid Features

* Categorical: Genre, category, brand; one-hot or learned embeddings via entity embedding layers
* Numerical: Price, duration, release year; normalise or bucket then embed
* Feature crosses: Explicit pairwise interactions between feature combinations; high signal for structured data
* Knowledge-graph attributes: Item properties from external KG such as Wikidata or product ontologies

3.2 Deep Content Models & Foundation Encoders

Item Representation with Foundation Models

* Text: E5-large, BGE-M3, or GTE-Qwen2 for item descriptions and reviews; mean pooling of token embeddings
* Image: DINOv2 ViT-L/14 or CLIP ViT-L for product images, movie thumbnails, album art
* Audio: Wav2Vec 2.0 or music-specialised encoders; Spotify and SoundCloud use waveform embeddings
* Video: VideoMAE frame sampling and temporal pooling; YouTube and TikTok thumbnail and content embeddings
* Zero-shot cross-modal: CLIP places image and text in the same space; text query drives ANN over image embeddings

YouTube DNN (Covington et al. 2016)

* Candidate generation: Average embeddings of watched video IDs and search tokens, then MLP, then softmax over all videos
* Ranking: Separate DNN predicting watch time (regression), not CTR; watch time aligns better with user value
* Extreme multi-class: Frame as classification over millions of videos for training efficiency
* Key lesson: Choosing the right proxy metric matters more than architectural sophistication

DSSM - Deep Structured Semantic Models (Microsoft 2013)

* Dual encoder for query-document matching; direct ancestor of the Two-Tower architecture
* Content features on the item tower: New items without interaction history still get quality embeddings

2026 TREND: Content-based models are experiencing a renaissance driven by foundation model encoders. A DINOv2 or E5-large encoder produces semantically rich item embeddings that generalise across domains with minimal fine-tuning, enabling strong cold-start recommendations.

4. Two-Tower Models

Two-Tower (Dual Encoder) models are the dominant architecture for large-scale candidate retrieval. Used by Google, Meta, Pinterest, Airbnb, and LinkedIn. The core idea: encode users and items into a shared embedding space and score with dot product or cosine similarity.

```
score(u,i) = f_user(x_u)^T * f_item(x_i)    where f_user and f_item are separate neural encoders
```

4.1 Architecture & Training Objectives

Architecture

* User tower: Takes user features (ID, interaction history, context) through MLP to produce d-dim embedding
* Item tower: Takes item features (ID, content, metadata) through MLP to produce d-dim embedding
* Shared embedding space: Both towers project to the same dimension and are scored by inner product
* Asymmetry: Item embeddings precomputed offline at scale; user embeddings computed online at query time
* L2 normalisation: Often applied to both towers to use cosine similarity; produces more stable training

Training Objectives

* Sampled softmax with in-batch negatives: Use other items in the batch as negatives; simple and scalable default

```
L = -log[ exp(s(u,i+)/tau) / SUM_{j in batch} exp(s(u,j)/tau) ]    tau = temperature
```

* BPR (Bayesian Personalised Ranking): Pairwise objective; positive must score higher than negative

```
L_BPR = -SUM_{(u,i,j)} log sigma(s(u,i) - s(u,j))    i = positive item, j = negative item
```

* Mixed Negative Sampling (MNS): Combine uniform and hard negatives; standard in production systems

Key Design Decisions

* Embedding dimension: Typically 64-256; larger improves recall but slows ANN search
* User history encoding: Average pooling vs transformer over interaction sequence; transformer wins for long histories
* Feature freshness: Item features updated daily; real-time user context recomputed online per request

4.2 Negative Sampling Strategies

Negative sampling is the single most important implementation detail in two-tower training. Negative quality determines model quality more than architecture choice.

Strategy	Mechanism	When to Use
Uniform random	Sample items uniformly from catalogue	Baseline; works with large catalogue
In-batch negatives	Other items in same training batch	Default; free; efficient at large batch sizes
Popularity-based	Sample proportional to item popularity	Corrects popularity bias in in-batch negatives

Hard negatives	Items user was exposed to but did not engage	Improves discrimination near decision boundary
Curriculum negatives	Start easy, progressively harder during training	Stable training; avoids early embedding collapse
Dynamic Neg. Sampling	Mine hardest negatives from current model top-k	Most powerful; risk of false negatives

CRITICAL: False-negative problem: An item not clicked because the user never saw it (not because they disliked it) may be treated as a negative. Mitigation: exclude known positives from the negative pool, or down-weight high-confidence negatives.

4.3 ANN Search & Production Deployment

Approximate Nearest Neighbour Index

* Precompute all item embeddings; index with FAISS, HNSW, or ScaNN
* At query time: encode user, then retrieve top-K items via ANN search in under 5 ms
* HNSW: Best recall/latency tradeoff for under 100M items; production default at most companies
* IVF-PQ: Product quantisation reduces memory; better for over 100M items with slight recall drop
* ScaNN (Google 2020): Anisotropic quantisation; best recall on billion-scale datasets

Training-Serving Consistency

* Problem: ANN retrieval at serving differs from exact search used during training
* Solution: Train with item sampling that mirrors the ANN retrieval distribution (MOBIUS, Baidu 2019)
* Embedding refresh: Item embeddings become stale; batch refresh daily or continuous streaming refresh

4.4 SOTA Two-Tower Systems

System	Company	Key Innovation	Study Takeaway
DSSM	Microsoft	First deep dual encoder for retrieval	Foundation; understand before everything else
YouTube DNN	Google	Watch-time prediction; extreme softmax	Proxy metric matters more than architecture
PinSage	Pinterest	GNN item tower with graph random walks	Graph structure dramatically improves item reps
MOBIUS	Baidu	Approximate inner product in training	Align training objective with ANN retrieval
EBR	Meta	Hard negative mining pipeline	Negative quality dominates model quality
Poly-encoders	Meta/HF	Multiple summary vectors per context	Better than single vector for complex queries

SE-GNN	Alibaba	Side information and graph structure	Item attributes plus graph equals strong cold-start
TIGER	Google	RQ-VAE item codes and generative retrieval	Discrete item codes enable LLM-based retrieval

5. Embedding-Based Recommendation

Embedding-based methods learn dense representations from item sequences, user interaction graphs, and session data. They bridge matrix factorization and deep learning by treating recommendation as a representation learning problem.

5.1 Item2Vec, Prod2Vec & Graph Embeddings

Item2Vec

Apply Word2Vec skip-gram to item sequences. Treat a user's consumption history as a sentence and items as words. Items appearing in similar contexts get similar embeddings.

```
L = SUM_t SUM_{-c<=j<=c, j!=0} log p(i_{t+j} | i_t)    p(i_j|i_t) proportional to
exp(v_j^T * u_t)
```

* Context window: Controls how far ahead and behind counts as context; typical range is 5-10
* Negative sampling: Sample k negatives per positive; update only sampled embeddings for efficiency
* Prod2Vec: Applied to purchase sequences; co-purchased items receive similar embeddings
* Spotify: Item2Vec with ANNOY tree for music recommendation at billion scale

Node2Vec & Graph Embeddings

* Node2Vec: Biased random walks on user-item graph; interpolate DFS community structure and BFS local neighbourhood
* DeepWalk: Pure random walks plus skip-gram; predecessor to Node2Vec
* LINE: Preserves first-order (direct) and second-order (shared neighbours) graph proximity
* Applications: Social graph preferences, co-purchase graph, co-view graph

5.2 Session-Based Recommendation

Session-based models focus on the current interaction sequence without requiring long-term user history. Critical for anonymous users and domains with natural session structure such as e-commerce.

GRU4Rec (Hidasi et al. 2016)

Apply GRU to item sequences. The final hidden state predicts the next item.

```
h_t = GRU(e_{i_t}, h_{t-1})    y_hat = softmax(W * h_t)
```

* Session-parallel mini-batches: Multiple sessions processed in parallel; handles variable length sequences
* TOP-1 and BPR-max losses: Specialised ranking losses outperform cross-entropy for next-item prediction
* GRU4Rec+: Data augmentation via sub-sequence training and dropout on item embeddings

SASRec (2018) - Self-Attentive Sequential

* Unidirectional transformer over item sequence; attends to all previous items in history
* Causal masking: Prevents attention to future positions; autoregressive at inference time
* Simpler than BERT4Rec; often competitive in quality; easier to deploy in production

BERT4Rec (2019)

Apply BERT-style masked language modelling to item sequences. Bidirectional attention captures both past and future context within a session.

```
L = -SUM_{i_m in masked} log P(i_m | i_unmasked)    mask 20% of items as cloze task
```

* Bidirectional context: Full attention over all items; better than unidirectional for offline evaluation
* Position embeddings: Learned positional encoding captures temporal ordering of interactions
* FMLP-Rec: Replace attention with FFT-based MLP; O(L log L) vs O(L^2); competitive quality
* Mamba4Rec (2024): Apply Mamba state space model to item sequences; linear complexity; strong performance

5.3 Graph Neural Networks for RecSys

LightGCN (He et al. 2020)

The most cited GNN recommendation model. Removes nonlinear activation and feature transformation from graph convolution, keeping only neighbourhood aggregation. Simplicity wins.

```
e_u^{k+1} = SUM_{i in N(u)} e_i^k / sqrt(|N(u)| * |N(i)|)    (same formula applies for items)
```

```
e_u^* = SUM_{k=0}^{K} alpha_k * e_u^k    (final embedding = weighted average across layers)
```

* Key insight: Linear propagation is sufficient; feature transformation hurts performance in pure CF
* Layer combination: Average across all K layers; captures both local and global graph structure
* Outperforms NGCF, GC-MC, and PinSage on standard benchmarks with a much simpler architecture

Extensions and Contrastive Learning

* NGCF: Adds MLP transformation at each layer; useful when node features are available
* UltraGCN: Skips graph convolution entirely; directly optimises graph proximity approximation; faster convergence
* SimGCL (2022): Graph contrastive learning; augment user-item graph and maximise agreement across views
* NCL (2022): Neighbour-enriched contrastive learning; structural and semantic neighbours used as positives

Knowledge Graph Integration

* Ripple Network: Propagate user preferences through KG ripples to discover item attribute chains
* KGCN: GCN over knowledge graph; aggregate relation-weighted entity embeddings for items
* KGAT: Graph attention over user-item and knowledge graph jointly; current KG-RecSys SOTA

PRACTICAL START: LightGCN is the recommended starting point for graph-based recommendation. It is simple to implement in a day, well-studied, and competitive with far more complex models. Add complexity only when you can attribute the improvement to a specific design choice.

6. LLM-Powered Recommendation

LLMs bring language understanding, world knowledge, and reasoning to recommendation systems. This is the fastest-moving frontier in the field, evolving from LLMs as feature encoders in 2022 to LLMs as autonomous recommendation engines in 2025-2026.

6.1 Prompting & In-Context Recommendation

Zero-Shot and Few-Shot Prompting

* Zero-shot: Provide user history in text and ask LLM to recommend next items; works but lacks fine-grained personalisation
* Few-shot: Include examples of similar users and their next interactions in the prompt context
* Chain-of-thought: Ask LLM to reason step-by-step about user preferences before producing the final list
* Role-play framing: 'You are a recommendation expert who knows this user deeply'; measurably improves quality

Cold-Start via In-Context Learning

* New user cold start: LLM generalises from item descriptions and minimal history using world knowledge
* Cross-domain transfer: LLM knowledge enables recommendations across domains such as music to podcasts without domain-specific training
* Constraint satisfaction: 'Vegetarian, budget-conscious, quick weeknight dinner'; LLM handles complex constraints naturally
* Explanation generation: LLM can explain why an item is recommended; extremely high business value

LLM as Ranker

* PairRank: Give LLM a pair of items; ask which is more relevant for this user; pairwise ranking signal
* ListRank (LLMRank): Provide list of candidates; ask LLM to reorder by relevance; listwise ranking
* GPT4Rec (2023): Generate hypothetical search queries from user history; retrieve using those queries as text

6.2 Fine-Tuned LLM Recommenders

P5 - Unified Text-to-Text Framework (Geng et al. 2022)

Frame all recommendation tasks as text-to-text. Rating prediction, sequential recommendation, explanation generation, and review summarisation all share a single T5 model.

```
Input: 'User_123 has interacted with item_456, item_789. Predict rating for item_234.'
```

* Multi-task: One model for retrieval, ranking, rating prediction, and explanation; shared representation
* Personalised prefix: User interaction history expressed as natural-language prefix
* Limitation: Item IDs as tokens are meaningless to the LLM; the tokeniser does not know item_234

Solving the Item ID Problem

* Item descriptions as IDs: Replace item_234 with 'The Dark Knight (2008), action, Christopher Nolan, 9.0 IMDb'
* Collaborative tokens (TALLRec): Learn special tokens for items via contrastive training; blends item semantics and CF signal
* RQ-VAE item codes: Encode items as discrete codes from residual-quantisation VAE; tokenisable sequences

* TIGER Generative Retrieval: Train LLM to autoregressively generate item codes given user history; no ANN index needed

Instruction Tuning for Recommendation

* TALLRec (2023): Instruction-tune LLaMA with recommendation-specific templates via LoRA; efficient adaptation
* BIGRec (2023): Grounded instruction-following; generation constrained to actual item catalogue
* InstructRec (2023): Diverse instruction templates covering personalised, situational, and intentional instructions
* RLMRec (2024): RL-aligned user preference modelling; DPO-style training on preference pairs

LLM Embeddings for Recommendation

* E5-Mistral and LLMEmbed (2025): LLaMA-3 backbone fine-tuned with contrastive objective; replaces Item2Vec
* Benefit: World-knowledge-enriched item embeddings; cross-domain transfer without retraining from scratch
* Challenge: 4096-dim embeddings are expensive to index and serve; quantise to 256-dim for ANN

6.3 SOTA LLM RecSys Models 2025-2026

Model	Base LLM	Key Contribution	Best For
P5	T5-base	Unified text2text for all rec tasks	Academic benchmarks; multi-task prototyping
M6-Rec	M6 (Alibaba)	Billion-parameter LLM for e-commerce	Product recommendation with rich attributes
TALLRec	LLaMA-7B	Efficient LoRA instruction tuning	Low-resource fine-tuning; fast adaptation
TIGER	T5	Generative retrieval via RQ-VAE codes	End-to-end LLM retrieval without ANN index
LLMRank	GPT-4 / Claude	Zero-shot listwise re-ranking	Re-ranking stage; cold start; explanations
GPT4Rec	GPT-4	Hypothetical query generation for retrieval	Bridging generative LLM and retrieval index
BiLLP	LLaMA-2	Bi-level learning: pretrain then finetune	Domain-specific deep personalisation
RLMRec	LLaMA-2	RL-aligned preference modelling	Preference alignment without human labels
LC-Rec (2024)	LLaMA-2	LLM and CF jointly trained end-to-end	Best of LLM world knowledge and CF signals
LLMEmbed (2025)	LLaMA-3	Contrastive embedding from LLM backbone	LLM-quality embeddings for ANN retrieval

PRODUCTION PATTERN: Dominant 2026 production pattern: LLMs handle cold start, explanation, and constraint satisfaction. Traditional two-tower and GNN handle warm users at scale. LLM re-

ranks the top-50 shortlist from retrieval. This hybrid is deployed at Pinterest, Airbnb, and Alibaba.

7. Hybrid & Multi-Stage Ranking

No single paradigm dominates across all scenarios. Production systems combine multiple approaches in a carefully designed pipeline that balances recall, precision, latency, and diversity.

7.1 Wide & Deep Learning (Google, 2016)

Combine a wide memorisation component with a deep generalisation component in a jointly trained model. The foundation of most industry ranking systems.

```
y_hat = sigma(w_wide * [x, phi(x)] + w_deep * a_deep + b)
```

* Wide component: Linear model on raw and crossed features; memorises specific feature co-occurrences
* Deep component: Embedding and MLP on sparse features; generalises to unseen feature combinations
* Joint training: Gradients flow through both paths simultaneously; avoids feature engineering tradeoffs
* Limitation: Wide component requires manual feature crosses; replaced by DCN in newer systems

7.2 Deep & Cross Network V2 (DCN-V2, Google 2021)

Replace the wide component with a Cross Network that automatically learns bounded-degree feature interactions.

```
x_{l+1} = x_0 circle (W_l * x_l + b_l) + x_l    (cross layer: explicit polynomial
feature interaction)
```

* Cross layer: Creates bounded-degree interactions between all feature pairs; no manual crossing required
* Matrix W instead of vector w: Captures richer interactions; currently deployed in production at Google
* Parallel structure: Cross network and deep MLP run in parallel; outputs concatenated for final prediction

7.3 DeepFM (Guo et al. 2017)

Combine FM second-order feature interactions with a deep MLP, sharing the same embeddings. Outperforms Wide & Deep because FM captures all pairwise interactions automatically.

```
y_hat = sigma(y_FM + y_DNN)    y_FM = w0 + SUM w_i x_i + SUM_{i<j} <v_i, v_j> x_i x_j
```

* Shared embedding: FM and DNN use the same embedding layer; joint optimisation, better representations
* xDeepFM: Compressed Interaction Network (CIN) for vector-level feature interactions; higher-order FM extension

7.4 DLRM (Meta, 2019)

Meta's production-scale recommendation model. Processes dense and sparse features separately then uses explicit dot-product interaction.

* Bottom MLP: Dense features (age, past CTR, session length) projected to dense embedding

* Sparse embeddings: Each categorical feature gets its own embedding table; scales to trillions of parameters
* Interaction layer: Dot products between all embedding pairs; explicit second-order interaction
* Top MLP: Concatenated interactions projected to final CTR or CVR prediction
* Scale: Largest embedding tables exceed 100GB and are distributed across hundreds of machines

7.5 Multi-Task Learning in Ranking

Production ranking models predict multiple targets simultaneously: CTR, CVR, dwell time, rating. Multi-task learning improves each via shared representations.

ESMM - Entire Space Multi-task Model (Alibaba)

```
CVR = CTR x CTCVR    (model CVR in full impression space, not click-biased space)
```

* Sample-selection bias: Training CVR on clicks only misses non-click impressions; ESMM corrects this
* Shared embedding: CTR and CVR towers share item and user embeddings; cross-task transfer learning

MMoE - Multi-gate Mixture of Experts

* Multiple expert networks: Each expert specialises in different aspects of user preference
* Task-specific gating: Each task learns which experts to weight; prevents task conflict and the seesaw effect
* PLE (Progressive Layered Extraction): Separate shared and task-specific experts per layer; stronger than MMoE

BUSINESS REALITY: In production, the ranking model typically has the highest ROI for improvement. A 0.1% CTR gain on a large-scale ranking model is worth hundreds of millions in annual revenue.

8. Multimodal Recommendation

Multimodal recommendation leverages images, text, audio, and video to enrich item representations. Critical for e-commerce, music streaming, and content platforms where visual and audio signals drive engagement.

8.1 Vision-Enhanced Recommendation

VBPR - Visual Bayesian Personalised Ranking (He & McAuley 2016)

Augment matrix factorization with CNN visual features. User latent factors include a visual dimension aligned with item visual features.

```
r_hat_ui = p_u^T q_i + theta_u^T f(I_i)    where f(I_i) = foundation model features of item image
```

* Separate visual and latent factors: User has a visual preference factor and a latent CF factor
* Fine-grained attention: Later models use attention over image regions to focus on product details
* CLIP zero-shot: Text description query to CLIP text embedding to ANN over CLIP image embeddings

Multimodal Two-Tower

* Item tower: Fuse ID embedding plus image embedding plus text embedding via attention or MLP
* Benefit: New items get quality embeddings from content even without any interaction history
* SigLIP: Better than CLIP for fine-grained product retrieval; uses sigmoid loss without global normalisation

8.2 Fusion Strategies

Fusion Type	Mechanism	Tradeoff
Early fusion	Concatenate raw features before any model	Simple; high-dimensional; modality imbalance risk
Late fusion	Separate encoders; combine final scores	Modular and easy to update; loses cross-modal interaction
Mid fusion (cross-attn)	Cross-attention between modality embeddings	Best quality; more complex to train and serve
Gated fusion	Learnable per-modality attention weights	Adaptive; down-weights noisy modalities
Contrastive fusion	Align modality embeddings via contrastive loss	Enables cross-modal retrieval natively

8.3 Self-Supervised Multimodal Models

* MMSSL (2023): Self-supervised cross-modal alignment signals combined with user-item interaction data
* BM3 (2023): Bootstrap multimodal contrastive learning; align views across image, text, and ID embeddings
* SLMRec (2023): Replace heavy encoders with lightweight alternatives without quality loss

* MoRec (2023): Items represented purely by content features without IDs; enables cross-platform recommendation

PRACTICAL INSIGHT: Key challenge in multimodal recommendation: ID-based CF signals are usually much stronger than content signals for warm users. Use content modalities to supplement cold-start and new items, not to replace CF for established users.

9. Evaluation: Metrics, Benchmarks & Pitfalls

9.1 Offline Ranking Metrics

Metric	Formula and Meaning	Strength	Weakness
Precision@K	*Relevant items in top-K divided by K*	Directly measures output quality	Ignores position within K
Recall@K	*Relevant in top-K divided by total relevant*	Measures catalogue coverage	Ignores ranking order
NDCG@K	*Graded relevance weighted by rank position*	Position-sensitive; handles graded labels	Requires relevance grades
HR@K (Hit Rate)	*1 if any relevant item in top-K*	Simple; widely used in literature	Binary; insensitive to rank
MRR	*1 divided by rank of first relevant item*	Rewards getting the right item high	Only looks at first relevant
AUC	*P(positive scored above negative)*	Pairwise; threshold-free	Does not capture rank position
Diversity (ILD)	*1 minus avg pairwise item similarity*	Detects filter bubble effects	May conflict with relevance
Coverage	*Unique items recommended divided by catalogue*	Measures long-tail exploration	May incentivise irrelevant items
Novelty	*Average minus log2(item popularity)*	Rewards non-obvious discovery	Hard to set target threshold

9.2 Online Metrics & A/B Testing

* CTR (Click-Through Rate): Clicks divided by impressions; most common but gameable with clickbait
* Conversion Rate: Purchases divided by impressions; higher signal; suffers from attribution lag
* Watch Time and Session Length: Deeper engagement signal than raw clicks; YouTube's primary metric
* Revenue Per User: Direct business metric; reliable only when the conversion funnel is instrumented cleanly
* Long-term retention: D7 and D30 return rate; most aligned with user value; longest feedback loop

WARNING: The metric trap: Optimising CTR increases clickbait. Optimising watch time can increase outrage. Optimising immediate conversion hurts long-term retention. Always measure multiple metrics in A/B tests and actively check for metric conflict.

9.3 Standard Benchmark Datasets

Dataset	Domain	Scale	Common Use
MovieLens-1M/20M	Movies	1M-20M ratings	CF, MF, deep learning baselines

Amazon Review (2023)	E-commerce	571M reviews	Content, cold start, LLM fine-tuning
Yelp	Restaurants	7M reviews	Hybrid systems, explanation generation
LastFM	Music	19M listening events	Session-based and sequential models
Gowalla / Foursquare	Location (POI)	1M+ check-ins	GNN, location-aware recommendation
Taobao / Tmall	E-commerce	100M+	Industrial-scale ranking, CTR prediction
KuaiRec	Short video	11M (fully observed)	Unbiased evaluation via dense matrix
MIND	News	1M users	News recommendation, content freshness
Steam	Gaming	7M users	Bundle recommendation, social graph
BookCrossing	Books	1M ratings	Cold start research (very sparse)

9.4 Evaluation Pitfalls

* Popularity bias: Popular items dominate test sets; models that recommend popular items score well regardless of personalisation quality
* Exposure bias: Users only rate items they were exposed to; observed data is not random, biasing offline evaluation
* Temporal leakage: If test interactions pre-date some training interactions; always evaluate strictly by time
* Protocol inconsistency: Leave-one-out vs 20% holdout produce very different numbers; specify and report both
* KuaiRec solution: Fully observed interaction matrix means every user rated every item; eliminates exposure bias entirely

10. Production Architecture

A recommendation system that works in a notebook is straightforward. One that serves 100M users at sub-100ms latency, handles cold start, degrades gracefully, and can be safely A/B tested requires careful architectural thinking at every layer.

10.1 Feature Engineering & Feature Store

* User features: Last 50 interactions with IDs and timestamps, categorical aggregates, demographics, real-time context including time, device, location, and session depth
* Item features: Static metadata such as category, brand, and price tier; dynamic stats such as CTR last 24h, conversion rate, and days since last sold
* Cross features: Historical user-item interaction statistics such as whether the user has bought from this brand before
* Feature store (Feast, Tecton, or homegrown): Serve precomputed features at low latency; prevent training-serving skew which is the most common silent production bug
* Real-time features: Kafka to Flink to Redis pipeline for session features with under 1 second latency

10.2 Candidate Generation at Scale

* Multiple retrievers in parallel: Personalised two-tower, item-item CF, trending, diversity retriever all run simultaneously
* Retrieval diversity: Each retriever serves a different purpose including personalised, popular, new, editorial, and contextual
* ANN infrastructure: Faiss on GPU for throughput, ScaNN for recall, HNSW for latency; choose based on SLO
* Embedding refresh: Item embeddings updated daily via batch or continuously via streaming; stale embeddings degrade quality significantly
* Inverted index fallback: BM25 and Elasticsearch alongside ANN for keyword-matching and structured filter queries

10.3 Ranking & Serving

* Feature assembly: Merge candidate list plus precomputed features plus real-time context into a single request to ranking model
* Cascaded scoring: Lightweight model in 1ms, heavier model on top-200 in 10ms, LLM re-ranker on top-50 in 50ms
* Model serving: Triton, TorchServe, or TF Serving with batch inference, model versioning, and canary rollout
* Embedding caching: Cache user embeddings for 30-60 seconds to avoid recomputing on every page load

10.4 Cold Start Strategies

Cold Start Type	Problem	Best Solutions
New User	Zero interaction history	Onboarding quiz; content-based from profile; popular items; LLM inference from demographics

New Item	No ratings or interaction data	Content embeddings (CLIP, E5); attribute-based ranking; editorial boost at launch
Both New	User and item both cold	Pure content-based; global popularity; LLM recommendation from description text
Context Shift	User in new context (gift shopping, travel)	Context-aware retrieval; session sub-model; explicit signal collection

10.5 Bias, Fairness & Feedback Loops

* Popularity bias: Long-tail items are under-recommended; use inverse-popularity weighting in training
* Position bias: Items in top positions get more clicks regardless of quality; correct with Inverse Propensity Scoring
* Filter bubble: Users increasingly see only what the algorithm predicts they will engage with; enforce diversity constraints in re-ranking
* Feedback loop amplification: Model serves items, items get interactions, model trains on interactions, initial biases amplify over time
* Demographic fairness: Ensure recommendation quality is equitable across demographic groups; measure per-group NDCG
* Content safety: Filter harmful, illegal, or policy-violating items from all recommendation surfaces

10.6 Production Monitoring

* Coverage drift: Alert if recommendation diversity drops; indicates feedback loop or filter issue
* Latency SLOs: p50 under 30ms, p95 under 100ms, p99 under 200ms; fall back to cheaper model if SLOs breach
* Embedding drift: Monitor distribution of user and item embeddings over time; sudden shift indicates data pipeline failure
* Score distribution: Monitor ranking score distribution; shift indicates concept drift or training data contamination

CORE CHALLENGE: The hardest production problem is not building the model. It is the feedback loop. Your model changes what users see, which changes what they click, which changes your training data. Correcting for this requires causal thinking, not just ML pattern-matching.

11. Full SOTA Model Comparison

A comprehensive reference of the most important recommendation models with their architectural category, key innovations, and deployment suitability.

Model	Category	Year	Benchmark	Key Strength
SVD / SVD++	Matrix Factorisation	2009	Netflix	Bias terms and implicit feedback via SVD++
ALS	Matrix Factorisation	2008	MovieLens	Distributed training; implicit feedback data
CDAE	Autoencoder CF	2016	ML-1M	Denoising for collaborative filtering
Mult-VAE	Variational AE	2018	ML-20M	Principled probabilistic CF with ELBO
RecVAE	Variational AE	2020	ML-20M	Alternating VAE training; SOTA autoencoder CF
NCF	Neural CF	2017	MovieLens	MLP over user-item embeddings
DSSM	Two-Tower	2013	Query-Doc	First deep dual encoder; CF retrieval foundation
YouTube DNN	Two-Tower	2016	YouTube	Watch-time prediction; extreme multi-class softmax
Wide and Deep	Hybrid Ranking	2016	Google Play	Memorisation and generalisation jointly trained
DeepFM	Hybrid Ranking	2017	Criteo	Automatic second-order feature interactions via FM
DCN-V2	Hybrid Ranking	2021	Criteo	Cross-network v2; currently deployed at Google
DLRM	Hybrid Ranking	2019	Criteo/Internal	Meta-scale distributed embedding tables
PinSage	GNN	2018	Pinterest	GraphSAGE item tower with graph random walks
LightGCN	GNN	2020	Gowalla/Yelp	Simplest GNN for CF; strong consistent baseline
SimGCL	GNN + Contrastive	2022	Yelp/LFM	Graph contrastive learning for RecSys
KGAT	Knowledge Graph	2019	Amazon	Graph attention over KG and user-item graph jointly
GRU4Rec	Sequential	2016	RSC15	RNN-based session recommendation; TOP-1 loss

SASRec	Sequential	2018	Amazon	Efficient self-attentive sequential recommendation
BERT4Rec	Sequential	2019	Amazon/Steam	Bidirectional transformer for item sequences
Mamba4Rec	Sequential	2024	Amazon	SSM for sequences; linear complexity
MMSSL	Multimodal	2023	Amazon	Self-supervised cross-modal alignment
BM3	Multimodal	2023	Amazon/TikTok	Bootstrap multimodal contrastive learning
MoRec	Multimodal	2023	Amazon	Pure content representations; cross-platform transfer
P5	LLM Text2Text	2022	Amazon	Unified T5 for all recommendation tasks
TALLRec	LLM Fine-Tuned	2023	Amazon	Efficient LoRA instruction tuning on LLaMA
TIGER	LLM Retrieval	2023	Amazon	Generative retrieval via RQ-VAE item codes
LLMRank	LLM Ranker	2023	MovieLens	GPT-4 zero-shot listwise re-ranker
LC-Rec	LLM plus CF	2024	Amazon/Yelp	LLM backbone jointly trained with CF signals
RLMRec	LLM plus RL	2024	Amazon	RL-aligned user preference modelling
LLMEmbed	LLM Embedding	2025	Amazon	LLaMA-3 contrastive item embeddings for ANN

12. Research Frontiers 2026

12.1 Conversational Recommendation

* Multi-turn dialogue: System asks clarifying questions to iteratively refine the candidate set
* KBRD: Knowledge-based conversational recommendation using KG to understand user utterances
* Critiquing: User says more like this but cheaper or less action more drama; natural preference revision
* LLM-native CRS: Fine-tune LLM on user-utterance and item-list pairs for end-to-end conversational rec

12.2 Causal Recommendation

Standard CF learns correlations. Causal recommendation asks what a user would like if you actually showed it to them. This requires moving beyond observational data.

* IPS (Inverse Propensity Scoring): Reweight interactions by inverse probability of item exposure; debiases popularity
* DR (Doubly Robust) estimator: Combine direct model and IPS; more stable than either alone
* Debiased matrix factorization: Explicitly model exposure mechanism and factor it out of preference estimation
* Causal embeddings: Learn representations that disentangle user preference from item popularity

12.3 Generative Recommendation

Directly generate item sequences rather than rank a fixed candidate set. Removes the retrieval-ranking separation entirely.

* TIGER: Items encoded as RQ-VAE discrete token sequences; LLM generates next item codes autoregressively
* GPTRec: GPT-style autoregressive generation of item IDs; treats recommendation as language generation
* Benefit: Single model for retrieval and ranking; captures complex sequential dependencies
* Challenge: Item vocabulary explosion; millions of items need efficient tokenisation via RQ-VAE

12.4 Reinforcement Learning for Long-Term Value

* MDP formulation: State equals user interest state, Action equals item to recommend, Reward equals engagement or retention
* DQN-based: Q-network maps user state to item Q-values; handles delayed rewards via off-policy training
* REINFORCE for RecSys: Policy gradient with user engagement as reward; optimises long-term user value
* Offline RL: Learn from logged data without exploration; CQL prevents over-optimism on out-of-distribution actions
* Simulation: KuaiSim and RecoGym as user model environments for safe offline RL training

12.5 Foundation Models for Recommendation

Training large-scale recommendation foundation models on massive cross-domain interaction data, then fine-tuning for specific applications, is the most significant trend of 2025-2026.

* UniSRec (2022): Universal sequential recommendation; transferable item representations across domains via contrastive learning
* RecFormer (2022): Item text to BERT to zero-shot transfer across recommendation domains
* LC-Rec (2024): LLM jointly trained with collaborative filtering; combines world knowledge and interaction data
* IDGenRec (2024): Generate item IDs from descriptions; bridges content-based and ID-based systems

12.6 Federated & Privacy-Preserving Recommendation

* Federated learning: Train on user devices; aggregate only gradients; raw user data never leaves the device
* Differential privacy: Add calibrated Gaussian noise to gradients or embeddings; formal epsilon-delta guarantees
* On-device SLMs: Phi-3-mini and Gemma-2B run entirely on device; complete privacy with acceptable recommendation quality
* Secure aggregation: Cryptographic protocols aggregate user updates without any single server seeing individual contributions

12.7 Key Open Problems

Open Problem	Description and Current Best Approach
Feedback loop debiasing	Model shapes what users see which shapes training data. Best current: IPS plus DR estimation plus counterfactual evaluation on KuaiRec.
Extreme cold start	Zero interactions for new user and new item. Best current: LLM inference from item description text and user profile text.
Long-tail recommendation	System focuses on popular items and ignores 80% of catalogue. Best current: exploration bonuses and long-tail-specific retrieval models.
Offline-to-online eval gap	Models winning offline eval fail online A/B tests. Best current: KuaiRec fully observed data plus causal evaluation.
Cross-domain transfer	Preferences from music applied to podcasts. Best current: UniSRec and MoRec content-based cross-domain transfer.
Real-time adaptation	Interests change within a session; batch models are stale. Best current: real-time feature streaming plus session sub-models.
Privacy vs personalisation	Deep personalisation requires user data; users increasingly opt out. Best current: federated learning and on-device SLMs.

The best recommendation systems are not the most sophisticated. They are the ones most tightly aligned with genuine user value.

Measure what matters. Build what ships. Iterate relentlessly.